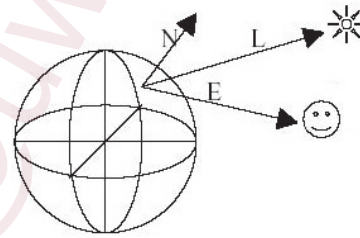


PART I. ILLUMINATION MODEL

1. When exposed to a distributed light (or a point light placed sufficiently far away), the illumination I at a surface point is determined by $\mathbf{N} \cdot \mathbf{L}$ and $\mathbf{E} \cdot \mathbf{R}$ in the equation,

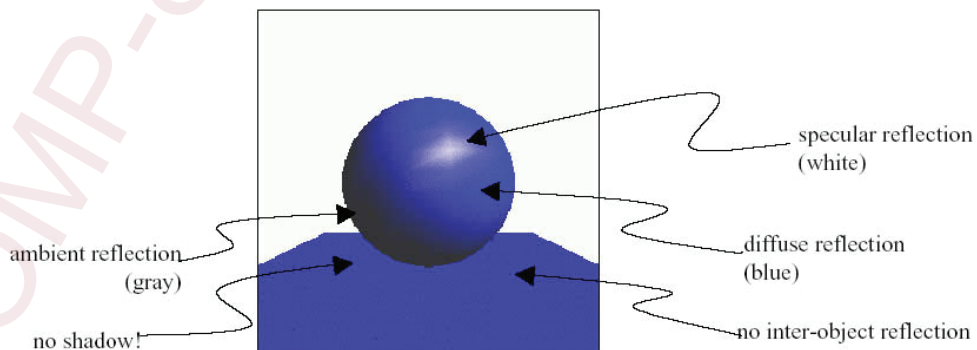
$$I = k_d I_a + \frac{I_p}{d + d_0} [k_d (\mathbf{N} \cdot \mathbf{L}) + k_s (\mathbf{E} \cdot \mathbf{R})^n]$$

- In the local illumination model:
 - $\mathbf{N}$, $\mathbf{L}$, $\mathbf{E}$, $\mathbf{R}$ are vectors defining the directions of surface normal, point light, viewer's eye, and reflection.
 - k_d and k_s are material constants
 - I_a and I_p are intensities of ambient and point light
 - d and d_0 are distance constants
 - $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \angle(\mathbf{a}, \mathbf{b})$



2. Lighting reflections

- *Ambient* reflection results from constant ambient light.
- *Diffuse* reflection is the normal reflection of a light source from an object.
- *Specular* reflections are the highlight reflections of a light source from an object in certain situations.



```

private Geometry geoTriangle(Color3f clr) {
    Point3f[] pts = {new Point3f(0f, 0f, 0.5f), new Point3f(0.5f, 0f, 0f),
        new Point3f(0f, 0.5f, 0f)};
    float u = (float) (1/Math.sqrt(3)); float[] nml = {u, u, u};
    TriangleArray tri = new TriangleArray(3, TriangleArray.NORMALS |
        TriangleArray.COORDINATES | TriangleArray.COLOR_3 );
    for (int i = 0; i < 3; i++) {
        tri.setColor(i, clr); tri.setCoordinate(i, pts[i]);
        tri.setNormal(0, nml);
    }
    return tri; }

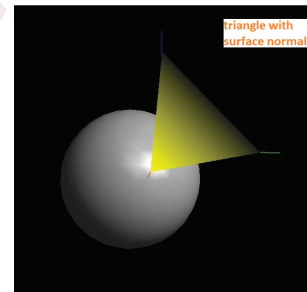
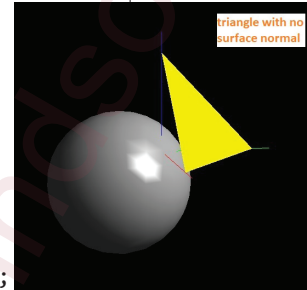
private void testLighting(TransformGroup scene_TG) {
    AppearanceExtra app_Extra = new AppearanceExtra();

    TransformGroup transTG = new TransformGroup();
    Transform3D translator = new Transform3D();
    translator.setTranslation(new Vector3f(-0.2f, -0.2f, -0.2f));
    transTG.setTransform(translator);
    transTG.addChild(app_Extra.materializeSphere(0.4f, 30));

    Geometry oneTriangle = geoTriangle(fun.Yellow);
    Appearance appTriangle = new Appearance();
    appTriangle.setMaterial(app_Extra.setMaterial());
    scene_TG.addChild(new Shape3D(oneTriangle, appTriangle));

    app_Extra.addLights(scene_TG, fun.White);
    scene_TG.addChild(transTG);
}

```



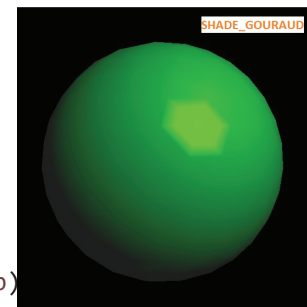
3. Java3D shading model

- The shade model is specified as one of SHADE_GOURAUD, SHADE_FLAT, FASTEST, NICEST.
 - In *flat shading*, all pixels for a polygon are assigned the shade value from one vertex of the polygon.
 - A curved surface is represented as a set of constantly shaded plane surfaces.
 - In *Gouraud shading*, each pixel is shaded with a value derived from trilinear interpolation of the shade value of each vertex of its enclosing polygon.
 - After the intensity values at the vertices of each polygon are determined, all other intensities for the surface are calculated from them.

```

public class AppearanceExtra {
    /* a function to add a point and an ambient light to scene_TG */
    public void addLights(TransformGroup scene_TG) {
        BoundingSphere bounds = new BoundingSphere(new Point3d(0.0, 0.0, 0.0), 100.0);
        AmbientLight amLgt = new AmbientLight(new Color3f(0.2f, 0.2f, 0.2f));
        amLgt.setInfluencingBounds(bounds);
        scene_TG.addChild(amLgt);
        Color3f clr = new Color3f(0.0f, 1.0f, 0.2f);
        Point3f pt = new Point3f(2.0f, 2.0f, 2.0f);
        Point3f atn = new Point3f(1.0f, 0.0f, 0.0f);
        PointLight ptLight = new PointLight(clr, pt, atn);
        ptLight.setInfluencingBounds(bounds);
        scene_TG.addChild(ptLight);
    }
    /* a function to create and return material definition */
    public Material setMaterial() {
    /* a function to create and return a sphere in 'radius' */
    public Sphere materializeSphere(float radius) {
        Appearance app = new Appearance();
        Color3f black = new Color3f(0f, 0f, 0f);
        ColoringAttributes ca = new ColoringAttributes(black,
            ColoringAttributes.SHADE_GOURAUD);
        // FASTEST NICEST SHADE_FLAT SHADE_GOURAUD
        app.setColoringAttributes(ca);
        app.setMaterial(setMaterial());
        return new Sphere(radius, Sphere.GENERATE_NORMALS, 30, app)
    }
}

```



PART II. LIGHTING

1. Java 3D provides four types of lights

Class Hierarchy

```

java.lang.Object
├── org.jogamp.java3d.SceneGraphObject
│   ├── org.jogamp.java3d.Node
│   │   ├── org.jogamp.java3d.Leaf
│   │   └── org.jogamp.java3d.Light
│       ├── org.jogamp.java3d.AmbientLight
│       ├── org.jogamp.java3d.DirectionallLight
│       └── org.jogamp.java3d.PointLight
│           └── org.jogamp.java3d.SpotLight

```

2. Creating ambient lights

- Light rays aim in all directions, flooding an environment and illuminating shapes evenly

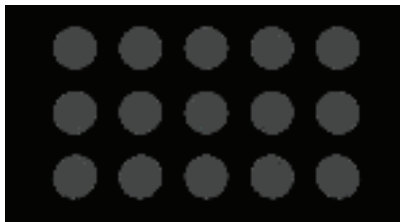
- AmbientLight example code

- Create a light

```
AmbientLight myLight = new AmbientLight( );  
myLight.setEnable( true );  
myLight.setColor( new Color3f( 1.0f, 1.0f, 1.0f ) );
```

- Set its influencing bounds

```
BoundingSphere myBounds  
    = new BoundingSphere(new Point3d( ), 1000.0 );  
myLight.setInfluencingBounds( myBounds );
```



3. Creating directional lights

- Parallel light rays aiming in a direction (defaultz-axis).

- DirectionalLight example code

- Create a light

```
DirectionalLight myLight = new DirectionalLight( );  
myLight.setEnable( true );  
myLight.setColor( new Color3f( 1.0f, 1.0f, 1.0f ) );  
myLight.setDirection( new Vector3f( 1.0f, 0.0f, 0.0f ) );
```

- Set its influencing bounds

```
BoundingSphere myBounds  
    = new BoundingSphere(new Point3d( ), 1000.0 );  
myLight.setInfluencingBounds( myBounds );
```

4. Creating point lights

- Light rays emit radially from a point in all directions
- Using point light attenuation
 - Point light rays are attenuated:
 - As distance increases, light brightness decreases
 - Attenuation is controlled by three coefficients:

$$l_{brt} = \frac{l_{int}}{c_0 + l_{lnr} * d + l_{qrt} * d^2}$$

- l_{brt} is the brightness to be calculated
- l_{int} is light intensity
- c_0 is a constant and d is distance
- l_{lnr} is a linear and l_{qrt} a quadratic multiplier

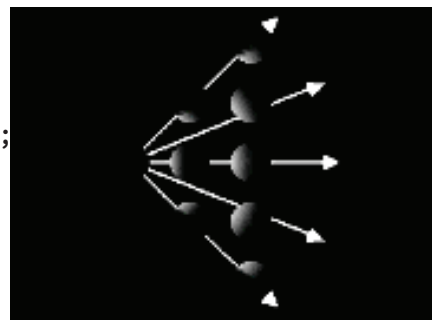
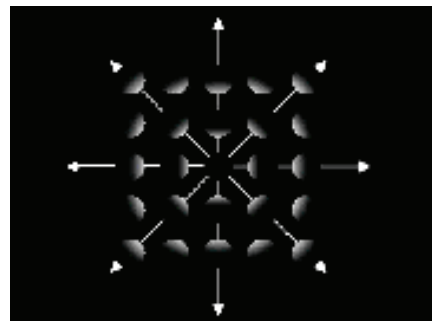
• PointLight example code

- Create a light

```
PointLight myLight = new PointLight( );
myLight.setEnabled( true );
myLight.setColor(new Color3f(1f, 1f, 1f));
myLight.setPosition(new Point3f(0f, 1f, 0f));
myLight.setAttenuation(new Point3f(1f, 0f, 0f));
```

- Set its influencing bounds

```
BoundingSphere myBounds
    = new BoundingSphere(new Point3d( ), 1000.0 );
myLight.setInfluencingBounds( myBounds );
```



5. Creating spot lights

- Light rays emit radially from a point, within a cone
 - Vary the spread angle to widen, or narrow the cone
 - Angle varies from 0.0 to $\text{PI}/2.0$ radians (default PI)
 - A value of PI radians makes the light a `PointLight`
 - Vary the concentration to focus the spot light
 - It varies from 0.0 (unfocused) to 128.0 (focused)
 - The default is 0.0
 - The default aim direction is (0.0, 0.0, -1.0).
- `SpotLight` example code
 - Create a light

```
SpotLight myLight = new SpotLight( );
myLight.setEnable( true );
```

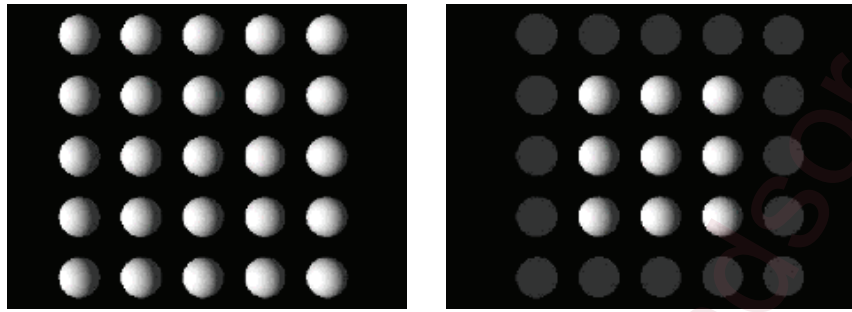
```
myLight.setColor( new Color3f( 1.0f, 1.0f, 1.0f ) );
myLight.setPosition(new Point3f( 0.0f, 1.0f, 0.0f));
myLight.setAttenuation(new Point3f(1.0f, 0.0f, 0.0f));
myLight.setDirection(new Vector3f(1.0f, 0.0f, 0.0f ));
myLight.setSpreadAngle( 0.785f ); // 45 degrees
myLight.setConcentration( 3.0f ); // Unfocused
```

- Set its influencing bounds

```
BoundingSphere myBounds
    = new BoundingSphere(new Point3d( ), 1000.0 );
myLight.setInfluencingBounds( myBounds );
```

6. Using light influencing bounds

- A light's illumination is bounded to a region of influence
 - Shapes within the region may be lit by the light



- Light bounding:
 - Enables controlled lighting in large scenes
 - Avoids over-lighting a scene when using multiple lights
 - Saves lighting computation time
- Creating influencing bounds
 - A light region of influence is a bounded volume:

- Sphere, box, polytope, or combination using Bounds
- To make a global light, use a huge bounding sphere
- By default, lights have no influencing bounds and illuminate nothing!
 - Common error: forgetting to set influencing bounds
- Anchoring influencing bounds
 - A light bounding volume can be relative to:
 - The light's coordinate system
 - Volume centered on light
 - As light moves, so does volume
 - A Bounding leaf's coordinate system
 - Volume centered on a leaf node somewhere else in scene graph

- As that leaf node moves, so does volume
- If light moves, volume does not
- Influencing bounds example code
 - Set bounds relative to the light's coordinate system


```
PointLight myLight = new PointLight( );
myLight.setInfluencingBounds( myBounds );
```
 - Or relative to a bounding leaf's coordinate system


```
TransformGroup myGroup = new TransformGroup( );
BoundingLeaf myLeaf = new BoundingLeaf( myBounds );
myGroup.addChild( myLeaf );
. . .
PointLight myLight = new PointLight( );
myLight.setInfluencingBoundingLeaf( myLeaf );
```

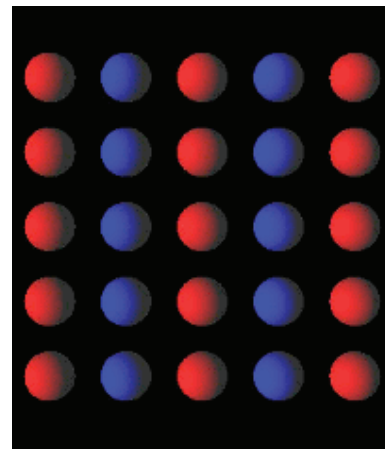
7. Scoping lights

- A light's illumination may be scoped to group(s) and only those shapes bounding in the group(s) are lit.
 - By default, lights have universal scope and illuminate everything within their influencing bounds
- Scoping example code
 - Build a group of shapes

```
TransformGroup myLightable=new TransformGroup();
Shape3D myShape = new Shape3D(myGeom, myAppear);
myLightable.addChild( myShape );
```

- Create a light and
add group to scope list

```
DirectionalLight myLight=new DirectionalLight();
myLight.addScope( myLightable );
```



8. Hints on Using Lights

- Use as few light sources as you can.
- Directional light sources are preferred since the computation required in rendering is significantly less than for point and spot lights.
- Point light sources are rarely used due to the high computational complexity.
- Including a single Ambient light source with a large region of influence is normal.
- The time required to include the Ambient light is small compared to other light sources.
- For objects, the more vertices, the smoother the lighting effect and the longer it will take to render.

PART III. FOG AND BACKGROUND

1. Fog and Background are two leaf nodes

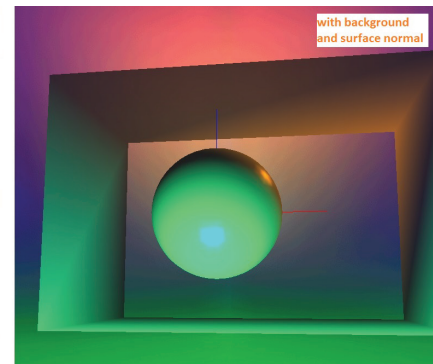
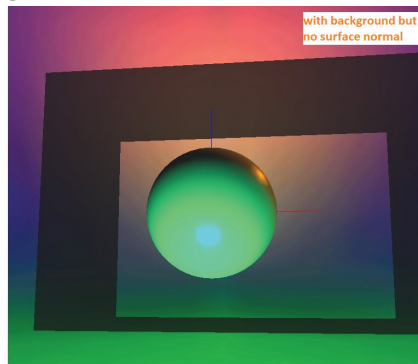
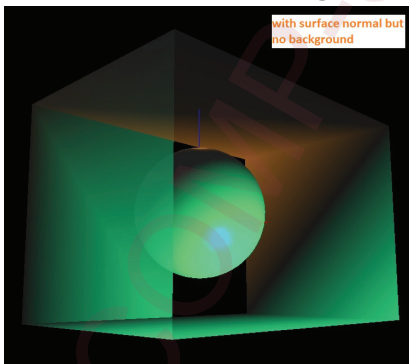
Class Hierarchy

```

java.lang.Object
├── org.jogamp.java3d.SceneGraphObject
│   ├── org.jogamp.java3d.Node
│   │   ├── org.jogamp.java3d.Leaf
│   │   │   ├── org.jogamp.java3d.Background
│   │   │   └── org.jogamp.java3d.Fog

```

2. Creating background



```

private Background createBkgnd(Color3f clr, BoundingSphere bounds) {
    Background bg = new Background();
    bg.setImage(new TextureLoader("images/back.jpg", null).getImage());
    // SCALE_NONE, SCALE_FIT_MIN, SCALE_FIT_MAX, SCALE_FIT_ALL
    bg.setImageScaleMode(Background.SCALE_FIT_MAX);
    bg.setApplicationBounds(bounds); // SCALE_REPEAT, SCALE_NONE_CENTER
    bg.setColor(clr);
    return bg;
}

private DirectionalLight directLgt(Color3f clr) {
    BoundingSphere bound = new BoundingSphere(new Point3d(0.0, 0.0, 0.0), 100.0);
    Vector3f d_vector = new Vector3f(0.5f, 0.5f, 0.0f); // pointing up-right
    DirectionalLight dir_light = new DirectionalLight(clr, d_vector);
    dir_light.setInfluencingBounds(bound);
    return dir_light;
}

private void createContent(TransformGroup scene_TG) {
    BoundingSphere bounds = new BoundingSphere(
        new Point3d(0.0, 0.0, 0.0), Double.MAX_VALUE);
    scene_TG.addChild(AppearanceTransparent.TransparentBox(true)); // add a box
    scene_TG.addChild(AppearanceExtra.materializeSphere(0.3f, 60)); // add a ball
    scene_TG.addChild(fun.threeAxes(fun.Blue, 0.5f)); // add axes
    AppearanceExtra.addLights(scene_TG, fun.Orange); // point+amb
    scene_TG.addChild(directLgt(fun.Lime)); // dir-light
    scene_TG.addChild(createBkgnd(fun.Grey, bounds)); // background
    scene_TG.addChild(createFog(fun.Grey, bounds)); // add fog
}

```

- Three types of backgrounds: color, image, geometry

- A Background node can set a single background color

- Fills canvas with the color

- Same color for all viewing directions and lighting

- Example code:

```

Background myBack = new Background( );
myBack.setColor( new Color3f( 0.3f, 0.0f, 0.0f ) );
BoundingSphere myBounds = new BoundingSphere(
    new Point3d( ), 1000.0 );
myBack.setApplicationBounds( myBounds );

```

- A Background node can set a background image

- Fills canvas with the image

- Image overrides background color

- Same image for all viewing directions and lighting

- A Background node can set background geometry
 - Geometry surrounds the viewer at "infinite"
 - The viewer can never move closer to the geometry
 - See different parts of the geometry when turning
 - The geometry is not lit by scene lights
 - Use background geometry to:
 - Build mountain, city skylines, or environment maps
 - Example code

```
// create background geometry in myBranch
BranchGroup myBranch = createBackground( );
Background myBack = new Background( );
myBack.setGeometry( myBranch );
BoundingSphere myBounds = new BoundingSphere(
    new Point3d( ), 1000.0 );
myBack.setApplicationBounds( myBounds );
```

3. Creating fog

- Fog affects shape color, not shape profile
 - Distant shapes have the fog color and crisp profiles
 - Need to set the background color to the fog color!
- There are two types of fogs
 - **ExponentialFog** uses $f_{xpl} = e^{-density \times z}$ to increase fog thickness exponentially with distance z , where $0 \leq density \leq 1$ is the density of the fog



Haze

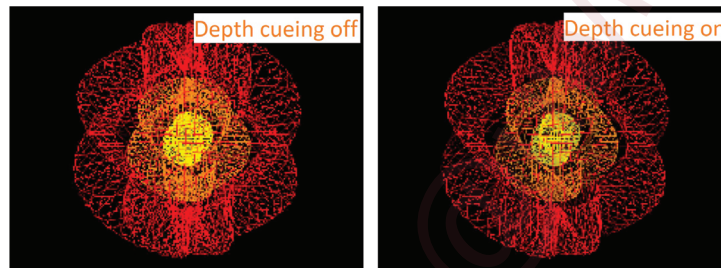


Light fog



Heavy fog

- **LinearFog** uses $f_{lnr} = (d_{back} - z) / (d_{back} - d_{front})$ to increase fog thickness linearly with distance z , where d_{front} and d_{back} are distance at which fog starts obscuring objects or totally obscures objects
 - Easier to control, though less realistic fog
 - By default, $d_{front} = 0$ and $d_{back} = 1$
 - For depth-cueing, use black linear fog
 - Set d_{front} and d_{back} to center or back of a shape



- **Common error:** forget to set influencing bounds

```
private static Fog createFog(Color3f clr, BoundingSphere bounds) {
    ExponentialFog e_fog = new ExponentialFog(clr, 0.6f);
    LinearFog l_fog = new LinearFog(clr, 1f, -1f);
    e_fog.setInfluencingBounds(bounds);
    l_fog.setInfluencingBounds(bounds);
    return l_fog; // e_fog l_fog
}
```

